```
                    if tot[1] >= tot[0] + tot[3]:           # base dominated by freezing: q*4 where it s
its
                        fz = con[:, 1]; o2 = np.argsort(-fz); c2 = np.cumsum(fz[o2]) / max(fz.sum(), 1e-
300)
                        ks = o2[:int(np.searchsorted(c2, frac)) + 1]; act = "q*4"
                    else:                                   # base dominated by the candidate itself (de
fect / jump)
                        dd = con[:, 0] + con[:, 3]; o2 = np.argsort(-dd); c2 = np.cumsum(dd[o2]) / max(d
d.sum(), 1e-300)
                        ks = o2[:int(np.searchsorted(c2, frac)) + 1]; act = "local Taylor" if repair els
e "stop"
            rec["new_cap"] = new_cap
            rec.update(terms=dict(zip(names, map(float, tot))), dominant=dom, steps90=int(len(ks)),
                       t_range90=[float(cert.tk[ks.min()]), float(cert.tk[ks.max() + 1])], action=act,
                       per_step=[float(v) for v in per])
        rec["every"] = cert.every; rec["q"] = cert.q.tolist(); rec["mu_max"] = [float(x["mu"].max()) for
 x in cert.d]
        rec["psi_cp"] = [float(np.max(cert.nb[c])) for c in cert.cps]
        hist.append(rec)
        if log: print(json.dumps({k: v for k, v in rec.items() if k not in ("bounds", "per_step", "q", "
mu_max", "psi_cp")}, default=float), file=log, flush=True)
        if best is None or tstar > best["t_star"]: best = rec
        if act is None or act == "stop" or r == rounds: break
        if act == "cap/10":
            if rec["new_cap"] <= 1e-8: break
            cert = Cert(S, cert.C, cap_rel=rec["new_cap"], q=cert.q, every=cert.every, e0_extra=cert.e0_
extra, hk=cert.hk)  # e0_extra already in step-0 norm
        elif act == "q*4":
            ks2 = [k for k in ks if cert.q[k] * 4 <= 1024]
            if not ks2: break
            for k in ks2: cert.q[k] *= 4
            cert.redo(ks2)
        else:
            new = [k for k in ks if k not in rep]
            if not new: break
            for k in new: rep.add(k); cert.C[k] = taylor_from(S, cert.C[k][0], cert.hk[k])
            cert.redo(new)
    return hist, best


def certify_best(S, C, **kw):
    kw = dict(kw); fast = kw.pop("fast", True); kw2 = kw
    """certify in two norms (time-local and global species scaling) and keep the longer proved interval;
    both are valid norms, so the better certificate is still a proof"""
    out = {}
    for rule in ("local", "global"):
        S.spec["sigma_rule"] = rule
        hist, best = certify(S, C, **kw2); best = dict(best); best["sigma_rule"] = rule
        out[rule] = (hist, best)
        if best["proved_full"] and fast: break        # already proved on [0,T]: the second norm is not
needed
    S.spec["sigma_rule"] = "local"
    key = max(out, key=lambda r: (out[r][1]["proved_full"], out[r][1]["t_star"], -(out[r][1]["eps"] or 1
e300)))
    hist, best = out[key]
    best["other_rule"] = {r: dict(t_star=out[r][1]["t_star"], proved_full=out[r][1]["proved_full"], eps=
out[r][1]["eps"]) for r in out}
    return hist, best
```

linear.py — 선형 사전 인증과 π·i 판정

```
"""linear: a priori certification of u' = A u in the norm ||u||_W = ||W u||_2, and the pi / i judgments.
  tau(theta, n)                   one-step truncation bound of S_n(hA) against e^{hA} (theta = h ||A||
< n + 2)
  lognorm(A, W)                   mu = lambda_max((B + B^T)/2), B = W A W^-1  (||e^{tA}|| <= e^{t mu})
  certified_steps(normA, T, n, eps, mu)   smallest N with N tau e^{N tau + T mu+} <= eps  (Propositions
2.1, 2.2)
  beta(normA, T, n, N, mu)        the promised bound including the rounding floor N (n + 1) u (diagnost
ic floor)
  taylor_steps(Aop, u, h, N, n)   N steps of the truncated series S_n
  ladder(n)                       remainder ladder eps_n = 1/(n! n) (Cantor tail of e)
  imag_interval(n)                largest y with |S_n(i y')| <= 1 for all |y'| <= y   (0 when unstable
at the origin)
  mod4(n)                         stable near the origin on the imaginary axis iff n = 0, 3 (mod 4)
  amp_phase(R, y)                 (|R|-1)^2 and 4|R| sin^2(dtheta/2): |R - e^{iy}|^2 split into length
and angle
  closure_defect(UOhat, omega, h, N, n)   modal prediction of the closure defect after N steps
  order_judge(errs, Ns, beta_ref, nominal)   orders from points with e_h > 10 beta_ref only (resolution
rule)
```

```
  identify_R(step, z)              amplification factor of a black-box one-step map from a scalar or rot
ation-block input"""
import math
import numpy as np

U = 2.0 ** -53


def tau(theta, n):
    return theta ** (n + 1) / math.factorial(n + 1) * (n + 2) / (n + 2 - theta)


def lognorm(A, W=None):
    B = A if W is None else W @ A @ np.linalg.inv(W); return float(np.max(np.linalg.eigvalsh(0.5 * (B +
B.T)))), float(np.linalg.norm(B, 2))


def certified_steps(normA, T, n, eps, mu=0.0):
    mup = max(mu, 0.0); N = max(1, math.ceil(normA * T / (n + 1)))
    while True:
        th = normA * T / N
        if th < n + 2:
            t = tau(th, n)
            if N * t < 50 and N * t * math.exp(N * t + T * mup) <= eps: return N, N * t * math.exp(N * t
 + T * mup)
        N = max(N + 1, int(N * 1.05))


def beta(normA, T, n, N, mu=0.0):
    t = tau(normA * T / N, n); return N * t * math.exp(N * t + T * max(mu, 0.0)) + N * (n + 1) * U


def taylor_steps(Aop, u, h, N, n=8):
    for _ in range(N):
        c = [u]
        for j in range(n): c.append(Aop(c[-1]) * h / (j + 1))
        u = sum(c)
    return u


def ladder(n): return 1.0 / (math.factorial(n) * n)


def Sn(z, n): return sum(z ** k / math.factorial(k) for k in range(n + 1))


def imag_interval(n, ymax=6.0, pts=200001):
    y = np.linspace(0, ymax, pts)[1:]; bad = np.abs(Sn(1j * y, n)) > 1 + 1e-14
    return 0.0 if bad[0] else float(y[np.argmax(bad)] if bad.any() else ymax)


def mod4(n): return n % 4 in (0, 3)


def amp_phase(R, y):
    dth = np.angle(R * np.exp(-1j * y)); return (np.abs(R) - 1) ** 2, 4 * np.abs(R) * np.sin(dth / 2) **
 2


def closure_defect(UOhat, omega, h, N, n=8):
    """modal prediction d_k = UO_k (S_n(-i omega_k h)^N - 1) for u_t + (transport) u = 0 with symbol i o
mega_k"""
    return UOhat * (Sn(-1j * omega * h, n) ** N - 1)


def order_judge(errs, Ns, beta_ref, nominal, tol=0.5):
    pts = [i for i in range(len(errs)) if errs[i] > 10 * beta_ref[i]]
    orders = [math.log(errs[i] / errs[i + 1]) / math.log(Ns[i + 1] / Ns[i]) for i in range(len(errs) - 1
) if i in pts and i + 1 in pts]
    if not orders: return orders, "proved=False, refuted=False (no resolvable points)"
    return orders, ("diagnostic: order deviates, look for a counterexample" if abs(orders[-1] - nominal)
 > tol else "diagnostic: order consistent")


def identify_R(step, z):
    """step(A, u, h) -> one step of the target scheme.  Real z: 1x1 input; complex z = a + ib: the rotat
ion block [[a,-b],[b,a]]"""
    if np.isreal(z):
        return complex(step(np.array([[float(np.real(z))]]), np.array([1.0]), 1.0)[0])
    a, b = float(np.real(z)), float(np.imag(z)); out = step(np.array([[a, -b], [b, a]]), np.array([1.0,
```

```
    0.0]), 1.0)
85      return complex(out[0], out[1])
```

verify.py — 시간 스킴의 에러 판정: 주장의 관계식과 반올림 상한, 방법 목록, 증폭 반례, 길이·각도 구간, 재판정

```
  1 """verify: does this time scheme have an error?  One rigorous procedure for linear and nonlinear systems
    , any scheme
  2 (one-step, multistep, implicit, adaptive, variable coefficients), any dimension, any order of discretiza
    tion (path A).
  3
  4 Every statement in the report is one of
  5   증명 (proof)          an inequality closed in the standard floating-point model (u = 2^-53, IEEE 754 r
    ound to nearest),
  6   반례 (counterexample) an inequality violated beyond a bound derived in the same model,
  7   정의 (definition)     an exact decomposition by a stated rule (the parts add up to the whole),
  8 and nothing else.  The scheme itself is a black box; what it must satisfy is stated by a CLAIM, i.e. a r
    elation
  9 together with the operations it is allowed to perform:
 10
 11   ("rk", A, b)                explicit Runge-Kutta tableau, any (fixed or adaptive) steps; the scheme ev
    aluates the stage
 12                               sums and F in floating point in any order
 13   ("lmm", alpha, beta, tau)   linear multistep with fixed step; tau = the residual the scheme promises f
    or its implicit
 14                               equation (0 for explicit methods)
 15   ("vbdf", qmax, tau)         variable-coefficient BDF of order <= qmax at every step, any step sizes, r
    esidual tau
 16
 17 Bounds used for a counterexample are derived from the claim itself (the operation count of the claimed f
    ormula, the
 18 rounding of F, a Jacobian enclosure on a box around the stages), so a counterexample needs no assumption
     about the
 19 black box beyond the claim.
 20
 21   Euler decomposition, the same for linear and nonlinear systems:
 22     e  : the reference is the truncated Taylor series S_n (Leibniz recursion), certified by the integral
    -inequality
 23          certificate (cert.certify); the scheme's error is enclosed in [measured - beta, measured + beta
    ]
 24     i  : on the frozen linear problem y' = J y (J at the worst state) the exact flow obeys ||e^{tJ}||_W
    <= e^{t mu},
 25          mu a rigorous log-norm bound; the scheme's linear run exceeding e^{t mu} ||y0|| + (its rounding
     bound) is a
 26          counterexample to its stability (an amplification the exact flow cannot have)
 27     pi : the error e = y - u splits exactly as ||e||^2 = (||y|| - ||u||)^2 + 4 ||y|| ||u|| sin^2(theta/2
    ) (length and
 28          angle; Proposition 6.1 for vectors); with u enclosed, both parts are enclosed and the larger on
    e is proved when
 29          the enclosures separate
 30   Integral inequality: the certificate of the scheme's own output splits its bound exactly by step and t
    erm (definition)."""
 31 import math
 32 import numpy as np
 33 import sympy as sp
 34 from core import I
 35 from rig import gam, lmax_sym_rig
 36
 37 U = 2.0 ** -53
 38 PADB = 1.0 + 1e-6          # pad on bounds evaluated in floating point (positive sums, relative error <<
     1e-6)
 39 EPS_AMP = 1e-10            # 'amplifies' means: beyond the exact-flow bound e^{t mu} by more than this r
    elative slack (a stated definition)
 40
 41
 42 # ------------------------------------------------------------------- rounding of F and a Jacobian enclos
    ure on a box
 43 def _r_opcount(S):
 44     """operation count of evaluating the reaction term's expression tree (Sys.R_ops), plus a margin of 4
    """
 45     return getattr(S, "R_ops", 0) + 4 if not hasattr(S, "_kr") else S._kr
 46
 47
 48 def F_round(S, y):
 49     """componentwise upper bound of |fl(F(y)) - F(y)| (F = D L u + R(u) + src evaluated in any order)"""
 50     m, N = S.m, S.N; u = np.abs(y.reshape(m, N)); absL = np.abs(S.L); kr = _r_opcount(S); out = []
 51     Ra = S.Rabs_gen(*u)
 52     for a in range(m):
```

```
        lin = abs(S.D[a]) * (absL @ u[a]); ra = np.asarray(Ra[a], float) * np.ones(N); sa = np.abs(S.src[a * N:(a + 1) * N])
        out.append(gam(N + 3) * lin + gam(kr) * ra + gam(3) * (lin + ra + sa))
    return np.concatenate(out) * PADB


def J_abs(S, lo, hi):
    """v -> an upper bound of |J(x)| v for all x in the box [lo, hi] (interval Jacobian of R, |D L| exactly)"""
    m, N = S.m, S.N; box = [I(lo[a * N:(a + 1) * N], hi[a * N:(a + 1) * N]) for a in range(m)]; absL = np.abs(S.L); mags = {}
    for a, b in S.Jnz:
        v = S.Jf[a][b](*box); mags[(a, b)] = v.mag() if isinstance(v, I) else np.abs(np.asarray(v, float)) * np.ones(N)
    def apply(v):
        vv = np.asarray(v, float).reshape(m, N); out = np.zeros((m, N))
        for a in range(m): out[a] += abs(S.D[a]) * (absL @ vv[a])
        for (a, b), mg in mags.items(): out[a] += mg * vv[b]
        return out.ravel() * PADB
    return apply


# ------------------------------------------------------------------ claimed relations, checked on the actual output
def rk_step_bound(S, A, b, y, h, F=None, dh=0.0):
    """my evaluation y_hat of one step of the tableau from y, and a componentwise bound B with |y_bb - y_hat| <= B for
    ANY other standard-model evaluation y_bb of the same formula from the same y (stage sums in any order, F rounded as
    F_round, coefficients rounded to the nearest double)"""
    F = F or S.F; A = np.asarray(A, float); b = np.asarray(b, float); s = len(b)
    X, K = [], []
    for i in range(s):
        Xi = y + h * sum(A[i, j] * K[j] for j in range(i)) if i else y.copy(); X.append(Xi); K.append(F(Xi))
    yh = y + h * sum(b[i] * K[i] for i in range(s))
    g = gam(s + 6); r = [1e-9 * (1 + np.abs(Xi)) for Xi in X]
    # the black box's own step may differ from h by dh (its t_{k+1} = fl(t_k + h), our h = fl(t_{k+1} - t_k)):
    # first-order sensitivity of the stage arguments to h, bounded with |K| and the Jacobian enclosure (below)
    Dh = []
    for _ in range(6):
        Jb = [J_abs(S, X[i] - r[i], X[i] + r[i]) for i in range(s)]
        D = []
        for i in range(s):
            Di = np.zeros_like(y)
            for j in range(i):
                if A[i, j] != 0: Di = Di + h * abs(A[i, j]) * (Jb[j](D[j]) + 2 * F_round(S, np.abs(X[j]) + r[j]))
            Di = Di + 2 * g * (np.abs(y) + h * sum(abs(A[i, j]) * np.abs(K[j]) for j in range(i))) if i else Di
            D.append(Di * PADB)
        Dh = []
        for i in range(s):
            Dh.append(sum(abs(A[i, j]) * (np.abs(K[j]) + h * Jb[j](Dh[j])) for j in range(i)) if i else np.zeros_like(y))
        D = [D[i] + dh * Dh[i] * PADB for i in range(s)]
        if all(np.all(D[i] <= r[i]) for i in range(s)): break
        r = [np.maximum(r[i], 2 * D[i]) for i in range(s)]
    else:
        return yh, np.full_like(y, np.inf)
    B = sum(h * abs(b[i]) * (Jb[i](D[i]) + 2 * F_round(S, np.abs(X[i]) + r[i])) for i in range(s)) \
        + 2 * g * (np.abs(y) + h * sum(abs(b[i]) * np.abs(K[i]) for i in range(s))) \
        + dh * sum(abs(b[i]) * (np.abs(K[i]) + h * Jb[i](Dh[i])) for i in range(s)) * PADB
    return yh, B * PADB


def check_rk(S, t, Y, A, b, F=None):
    """first step k where |y_{k+1} - RK(y_k, h_k)| exceeds the bound (counterexample) -> (k, excess), or the number of
    steps checked (an int) when none does; the check stops at the first non-finite value"""
    nchk = 0
    for k in range(len(t) - 1):
        if not (np.all(np.isfinite(Y[k])) and np.all(np.isfinite(Y[k + 1]))): return nchk
        with np.errstate(all="ignore"):
            hk_ = t[k + 1] - t[k]; dh = 2 * U * (abs(t[k + 1]) + abs(hk_))
            yh, B = rk_step_bound(S, A, b, Y[k], hk_, F, dh)
            ex = np.abs(Y[k + 1] - yh) - B
        if not np.all(np.isfinite(yh)) or not np.all(np.isfinite(B)): return nchk
```

```
        if np.any(ex > 0): return k, float(np.max(ex)), [int(j) for j in np.argsort(-ex)[:3] if ex[j] > 0]
        nchk += 1
    return nchk


def check_lmm(S, t, Y, alpha, beta, tau=0.0, F=None):
    """fixed-step linear multistep relation  sum alpha_i y_{n+i} / h - sum beta_i F(y_{n+i}) = 0  (up to tau(1 + |F|)
    for an implicit scheme); the bound covers the rounding of both evaluations of the relation"""
    F = F or S.F; al, be = np.asarray(alpha, float), np.asarray(beta, float); k = len(al) - 1; hk = np.diff(t)
    if len(hk) == 0 or np.max(np.abs(hk - hk[0])) > 8 * U * t[-1]: return "n/a"
    h = hk[0]; g = gam(2 * k + 8); nchk = 0
    with np.errstate(all="ignore"): Fv = [F(y) for y in Y]
    for n in range(len(Y) - k):
        win = Y[n:n + k + 1]; fw = Fv[n:n + k + 1]
        if not all(np.all(np.isfinite(v)) for v in list(win) + list(fw)): return nchk
        r = sum(al[i] * win[i] for i in range(k + 1)) / h - sum(be[i] * fw[i] for i in range(k + 1))
        mag = sum(abs(al[i]) * np.abs(win[i]) for i in range(k + 1)) / h + sum(abs(be[i]) * np.abs(fw[i]) for i in range(k + 1))
        dh = 2 * U * (abs(t[n + k]) + h) + abs(t[n + k] - t[n] - k * h)        # the black box's h vs ours
        B = 2 * g * mag + 2 * sum(abs(be[i]) * F_round(S, win[i]) for i in range(k + 1)) + (tau * (1 + np.max(np.abs(fw[-1]))) if be[-1] != 0 else 0.0) \
            + sum(abs(al[i]) * np.abs(win[i]) for i in range(k + 1)) * dh / h ** 2 * 2
        ex = np.abs(r) - B * PADB
        if np.any(ex > 0): return n + k, float(np.max(ex)), [int(j) for j in np.argsort(-ex)[:3] if ex[j] > 0]
        nchk += 1
    return nchk


def _dweights(ts):
    ts = np.asarray(ts, float); n = len(ts); x = ts[-1]; w = np.zeros(n)
    for i in range(n):
        if i == n - 1: w[i] = sum(1.0 / (x - ts[k]) for k in range(n) if k != i)
        else: w[i] = np.prod([x - ts[k] for k in range(n) if k not in (i, n - 1)]) / np.prod([ts[i] - ts[k] for k in range(n) if k != i])
    return w


def check_vbdf(S, t, Y, qmax=5, tau=0.0, F=None):
    """variable-coefficient BDF: at every step some order q <= qmax satisfies p_q'(t_{n+1}) = F(y_{n+1})  (p_q the
    interpolant through the last q + 1 points) up to tau(1 + |F|) plus the rounding of the weights and sums.
    Returns (None, orders) or ((step, excess), orders)."""
    F = F or S.F; orders = []
    for n1 in range(1, len(t)):
        if not np.all(np.isfinite(Y[n1])): break
        fv = F(Y[n1]); best, bq = np.inf, None
        if not np.all(np.isfinite(fv)): break
        for q in range(1, min(qmax, n1) + 1):
            w = _dweights(t[n1 - q:n1 + 1]); r = sum(w[i] * Y[n1 - q + i] for i in range(q + 1)) - fv
            mag = sum(abs(w[i]) * np.abs(Y[n1 - q + i]) for i in range(q + 1))
            B = 2 * gam(4 * q + 10) * mag + 2 * F_round(S, Y[n1]) + gam(4) * np.abs(fv) + tau * (1 + np.max(np.abs(fv)))
            exv = np.abs(r) - B * PADB; ex = float(np.max(exv))
            if ex < best: best, bq, bv = ex, q, exv
        if best > 0: return (n1, best, [int(j) for j in np.argsort(-bv)[:3] if bv[j] > 0]), orders
        orders.append(bq)
    return None, orders


# ------------------------------------------------------------------- the candidate library (identification by elimination)
def _tab(rows, b):
    s = len(b); A = np.zeros((s, s))
    for i, r in enumerate(rows): A[i, :len(r)] = r
    return A, np.array(b, float)


def library(extended=False):
    from scipy.integrate._ivp.rk import RK45, RK23
    A45 = np.zeros((6, 6)); A45[:, :5] = RK45.A
    L = {"오일러": ("rk",) + _tab([[]], [1.0]),
         "호인(RK2)": ("rk",) + _tab([[], [1.0]], [0.5, 0.5]),
         "중점(RK2)": ("rk",) + _tab([[], [0.5]], [0.0, 1.0]),
         "SSP-RK3": ("rk",) + _tab([[], [1.0], [0.25, 0.25]], [1 / 6, 1 / 6, 2 / 3]),
```

```
         "RK4": ("rk",) + _tab([[], [0.5], [0, 0.5], [0, 0, 1.0]], [1 / 6, 1 / 3, 1 / 3, 1 / 6]),
         "보가키-샴핀 3": ("rk",) + _tab([[], [0.5], [0, 0.75]], [2 / 9, 1 / 3, 4 / 9]),
         "도르만-프린스 5(RK45)": ("rk", A45, np.array(RK45.B))}
    ab = {1: [1], 2: [-1 / 2, 3 / 2], 3: [5 / 12, -16 / 12, 23 / 12], 4: [-9 / 24, 37 / 24, -59 / 24, 55 / 24]}
    for k, c in ab.items():
        L[f"애덤스-배시퍼스 {k}"] = ("lmm", [0] * (k - 1) + [-1, 1], c + [0], 0.0)
    bdf = {1: ([-1, 1], 1), 2: ([1 / 3, -4 / 3, 1], 2 / 3), 3: ([-2 / 11, 9 / 11, -18 / 11, 1], 6 / 11),
           4: ([3 / 25, -16 / 25, 36 / 25, -48 / 25, 1], 12 / 25), 5: ([-12 / 137, 75 / 137, -200 / 137, 300 / 137, -300 / 137, 1], 60 / 137)}
    for k, (a_, b_) in bdf.items():
        L[f"BDF{k}(고정 걸음)"] = ("lmm", a_, [0] * k + [b_], "tau")
    L["가변 걸음 BDF(차수 ≤ 5)"] = ("vbdf", 5, "tau")
    if extended: L.update(library_open())
    return L


def library_open():
    """공개 목록(NodePy, Ketcheson et al.)에서 가져온 방법들: 명시적 룽게-쿠타 표 전부(내장 쌍은 b와 bhat 둘 다),
    애덤스-배시퍼스 1-8, 애덤스-몰턴 1-6(암시적, 잔차 tau), BDF 1-6, 뉘스트룀 2-6, 밀른-심프슨 2-6.
    계수는 NodePy의 정확한 유리수를 배정밀도로 옮긴 것이다(대상 코드도 배정밀도 계수를 쓴다)."""
    import warnings
    warnings.filterwarnings("ignore")
    import nodepy.runge_kutta_method as RKm, nodepy.linear_multistep_method as LMm
    out = {}
    for nm, m in RKm.loadRKM("All").items():
        try:
            A = np.array(m.A, dtype=float); b = np.array(m.b, dtype=float)
        except Exception:
            continue
        if np.any(np.triu(A) != 0): continue                        # 암시적 룽게-쿠타는 단계 방정식의 잔차 주장이 필요해 뺀다
        out[f"NodePy {nm}"] = ("rk", A, b)
        bh = getattr(m, "bhat", None)
        if bh is not None:
            try: out[f"NodePy {nm}(내장 bhat)"] = ("rk", A, np.array(bh, dtype=float))
            except Exception: pass
    fam = [("애덤스-배시퍼스", LMm.Adams_Bashforth, range(1, 9), 0.0), ("애덤스-몰턴", LMm.Adams_Moulton, range(1, 7), "tau"),
           ("BDF", LMm.backward_difference_formula, range(1, 7), "tau"), ("뉘스트룀", LMm.Nystrom, range(2, 7), 0.0),
           ("밀른-심프슨", LMm.Milne_Simpson, range(2, 7), "tau")]
    for fn, f, ks, tau in fam:
        for k in ks:
            try:
                m = f(k); al = [float(v) for v in m.alpha]; be = [float(v) for v in m.beta]
            except Exception:
                continue
            out[f"NodePy {fn} {k}"] = ("lmm", al, be, tau if be[-1] != 0 else 0.0)
    return out


def check_claim(S, t, Y, claim, tau=0.0, F=None):
    """-> (status, info): status 'refuted' (info = (step, excess)), 'consistent' (info = (steps checked, orders or None)),
    or 'n/a' (the relation cannot be applied: uneven steps for a fixed-step method, or no step could be checked)"""
    kind = claim[0]
    if kind == "rk":
        res = check_rk(S, t, Y, claim[1], claim[2], F)
    elif kind == "lmm":
        tt = tau if claim[3] == "tau" else float(claim[3]); res = check_lmm(S, t, Y, claim[1], claim[2], tt, F)
        if res == "n/a": return ("n/a", None)
    elif kind == "vbdf":
        tt = tau if claim[2] == "tau" else float(claim[2]); r_, orders = check_vbdf(S, t, Y, claim[1], tt, F)
        if r_ is not None: return ("refuted", r_)
        return ("consistent", (len(orders), orders)) if orders else ("n/a", None)
    else:
        raise ValueError(kind)
    if isinstance(res, tuple): return ("refuted", res)
    return ("consistent", (res, None)) if res > 0 else ("n/a", None)


# ------------------------------------------------------------------ the verification
def verify(S, solver=None, step=None, N=None, claim=None, claim_name="주장한 방법", eps=1e-6, tau=0.0, name="대상 스킴",
           n_ref=8, max_ref=4096, factory=None, top=3, use_library=True, space=None, extended_library=Fa
```

```
lse):
    """solver(F, y0, T) -> (t, Y), or step(F, y, h) with N uniform steps.  factory(param) -> solver for the prescription
    re-check (param = number of steps for a step function, or a tolerance factor).  Returns the report dict."""
    from semidisc import cand_from_values, cand_taylor
    from cert import certify
    from six_w import _plain_level, _horner_at
    if solver is None:
        def solver(F, y0, T, _N=N):
            Y = [y0.copy()]
            for _ in range(_N): Y.append(np.asarray(step(F, Y[-1], T / _N), float))
            return np.linspace(0, T, _N + 1), np.array(Y)
        if factory is None: factory = lambda n_: (lambda F, y0, T: solver(F, y0, T, n_))
    m, Nn = S.m, S.N; d = m * Nn; T = S.T; sw = S.sw; F = S.F; rep = {}
    # ---------------- 검사 순서의 첫 단계: 공간(space = dict(kind=주장한 공간 이산, x=마디 위치)); 반례면 여기서 멈춘다
    space_line = None
    if space is not None:
        import space_check as SC
        rs = SC.check(S.L, space["x"], space.get("kind"))
        if not rs["ok"]:
            fx = SC.check(SC.patched(S.L, rs["viol"]), space["x"], space.get("kind"))
            rep = SC.report(rs, space["x"], "공간 연산자 L", fx)
            rep["판정"]["text"] += ". 시간 스킴과 정확도의 검사는 공간이 통과한 뒤에 한다(검사 순서: 공간 → 시간 → 정확도)"
            return rep
        space_line = f"공간: {space.get('kind') or '주장 없음'}의 관계식·상수 보존·1차 보존이 {rs['n']}개 마디 모두에서 성립(증명)"
    t, Y = solver(F, S.u0.copy(), T); t = np.asarray(t, float); Y = np.asarray(Y, float); NS = len(t) - 1
    finite = bool(np.all(np.isfinite(Y)))
    # ---------------- 누가: the claim and the library, checked on the actual output (rigorous both ways)
    who_lines = []; refuted_claim = False; consistent = []; impl_where = None
    if claim is not None:
        with np.errstate(all="ignore"):
            st, info = check_claim(S, t, Y, claim, tau)
        if st == "refuted":
            refuted_claim = True; k_, ex = info[0], info[1]; bad_nodes = info[2] if len(info) > 2 else []
            who_lines.append(f"{claim_name}: 반례 — 걸음 {k_}(t = {t[min(k_, NS)]:.4g})에서 관계식이 공식에서 유도한 반올림 상한을 {ex:.2e} 넘는다(구현 에러)")
            impl_where = (k_, bad_nodes)
        elif st == "consistent":
            consistent.append(claim_name)
            nchk, ords = info; full = _expected_checks(claim, NS)
            who_lines.append(f"{claim_name}: 검사한 {nchk}걸음 모두에서 관계식이 반올림 상한 안에서 성립(증명)" + ("" if nchk >= full else f", 검사할 수 있는 {full}걸음 중 값이 유한한 앞부분") + (f", 걸음마다 차수 {_orders_txt(ords)}" if ords else ""))
        else:
            who_lines.append(f"{claim_name}: 걸음이 고르지 않아 고정 걸음 관계식을 적용할 수 없음")
    if use_library:
        out, cons = [], []
        for nm, cl in library(extended_library).items():
            with np.errstate(all="ignore"):
                st, info = check_claim(S, t, Y, cl, tau)
            if st == "consistent": cons.append(nm + (f"({_orders_txt(info[1])})" if info[1] else "") + ("" if info[0] >= _expected_checks(cl, NS) else f"[앞 {info[0]}걸음]"))
            elif st == "refuted": out.append(nm)
        consistent += cons
        who_lines.append("방법 목록에서 반례로 배제되지 않은 것: " + (", ".join(cons) if cons else "없음") + f" (배제 {len(out)}개)")
    if space_line: who_lines.insert(0, space_line)
    rep["누가"] = dict(status="반례" if refuted_claim else "증명", text="; ".join(who_lines) if who_lines else "주장도 목록도 검사하지 않음")

    # ---------------- e + integral inequality: certified reference, refined until every verdict closes
    def _reference(nref_, K_):
        Cr_ = cand_taylor(S, nref_, K=K_); _, br_ = certify(S, Cr_, rounds=2)
        if not br_["proved_full"]: return None
        hr_ = T / nref_
        refv_ = np.array([_horner_at(Cr_[min(int(tt / hr_), nref_ - 1)], tt - min(int(tt / hr_), nref_ - 1) * hr_) for tt in t])
        def _hround(tt):      # rounding of evaluating the reference polynomial by Horner at tt (standard model)
            k_ = min(int(tt / hr_), nref_ - 1); c_ = Cr_[k_]; s_ = tt - k_ * hr_
            return float(np.linalg.norm(sw * gam(2 * len(c_) + 4) * sum(np.abs(c_[j]) * abs(s_) ** j for j in range(len(c_)))))
        beta_ = np.array([_plain_level(br_, tt) + _hround(tt) for tt in t]) * PADB
```

```
        with np.errstate(all="ignore"):
            err_ = np.array([float(np.linalg.norm(sw * (Y[k] - refv_[k]))) for k in range(NS + 1)]); err_[~np.isfinite(err_)] = np.inf
        # the measured norm itself is rounded: |fl(||v||) - ||v||| <= gam(d + 4) ||v||
        lo_ = np.maximum(err_ * (1 - gam(d + 6)) - beta_, 0); hi_ = err_ * (1 + gam(d + 6)) + beta_
        return dict(Cr=Cr_, br=br_, nref=nref_, K=K_, hr=hr_, refv=refv_, beta=beta_, err=err_, lo=lo_, hi=hi_)
    def _open(R_):            # does some verdict still not close with this reference?  (accuracy verdict, or pi at the worst time)
        und = not np.any(R_["lo"] > eps) and not np.all(R_["hi"] <= eps)
        kw_ = int(np.argmax(np.where(np.isfinite(R_["err"]), R_["err"], -1)))
        if np.isfinite(R_["err"][kw_]) and R_["err"][kw_] > 0:
            Lp_, Ap_ = _pi_split(sw * Y[kw_], sw * R_["refv"][kw_], R_["beta"][kw_], R_["err"][kw_], d)
            und = und or not (Lp_[0] > Ap_[1] or Ap_[0] > Lp_[1])
        return und
    J0 = S.Jnum(S.u0.reshape(m, Nn)); nref = max(8, math.ceil(T * np.linalg.norm(J0, 2))); R = None
    while nref <= max_ref:
        R1 = _reference(nref, n_ref)
        if R1 is not None:
            R = R1
            if _plain_level(R["br"], T) <= 1e-2 * eps: break
        nref *= 2
    if R is not None:
        for _ in range(6):    # refine (more steps, higher Taylor order) while a verdict is still open
            if not _open(R): break
            R2 = _reference(min(2 * R["nref"], 4 * max_ref), min(R["K"] + 2, 16))
            if R2 is None or float(R2["beta"].max()) >= float(R["beta"].max()): break
            R = R2
    if R is None:
        if refuted_claim:
            rep["판정"] = dict(status="반례", text="구현 에러: 대상은 주장한 방법이 아니다(반례)")
            k_, bn = impl_where
            rep["어디서"] = dict(status="반례", text=f"구현 에러가 처음 드러난 자리: 걸음 {k_}(t = {t[min(k_, NS)]:.4g}), 성분·마디 " + ", ".join(f"성분 {j // Nn} 마디 {j % Nn}" for j in bn))
        else:
            rep["판정"] = dict(status="정의", text="범위 밖: 이 문제의 기준해는 적분 부등식으로 인증되지 않는다(가정 H 밖, 15절). 정확도 판정의 대상이 아니다")
        return rep
    Cr, br, nref, hr, refv, beta, err, lo, hi, K_ref = R["Cr"], R["br"], R["nref"], R["hr"], R["refv"], R["beta"], R["err"], R["lo"], R["hi"], R["K"]
    rep["무엇을"] = dict(status="증명", text=f"t = T에서 오차는 [{lo[-1]:.3e}, {hi[-1]:.3e}] 안에 있다(기준해: 테일러 S_{K_ref} {nref}걸음, 적분 부등식으로 인증 β = {beta[-1]:.1e})")
    k_ok = 0
    while k_ok <= NS and hi[k_ok] <= eps: k_ok += 1
    bad = np.where(lo > eps)[0]; t_bad = float(t[bad[0]]) if len(bad) else None
    rep["언제"] = dict(status="증명" if t_bad is None else "반례",
                       text=((f"[0, {t[k_ok - 1]:.4g}]에서 오차 ≤ {eps:g}가 증명된다" if k_ok > 1 else f"첫 걸음부터 오차 ≤ {eps:g}가 증명되지 않는다")
                             + (f"; t = {t_bad:.4g}에서 처음으로 오차 > {eps:g}가 반례로 확정된다" if t_bad is not None else "")))
    # ---------------- 어디서: component shares of the error vector, enclosed (definition + proof)
    kw = int(np.argmax(np.where(np.isfinite(err), err, -1)))
    wh = []
    if np.isfinite(err[kw]) and err[kw] > 0:
        e_mid = sw * (Y[kw] - refv[kw]); rad = beta[kw] + 4 * U * (np.abs(sw * Y[kw]) + np.abs(sw * refv[kw]))   # + rounding of the difference
        sq_lo = np.maximum(np.abs(e_mid) - rad, 0) ** 2; sq_hi = (np.abs(e_mid) + rad) ** 2
        tot_lo, tot_hi = max(float(np.sum(sq_lo)), 1e-300), float(np.sum(sq_hi))
        order = np.argsort(-np.abs(e_mid))[:top]
        wh.append(f"오차가 가장 큰 t = {t[kw]:.4g}에서 오차 제곱의 몫(구간): " + ", ".join(
            f"성분 {j // Nn} 마디 {j % Nn} [{sq_lo[j] / tot_hi:.2f}, {min(1.0, sq_hi[j] / tot_lo):.2f}]" for j in order))
    if not finite:
        kd = int(np.argmax(~np.isfinite(err))); wh.append(f"출력이 t = {t[kd]:.4g}에서 유한하지 않다")
    bs = {}
    if finite and NS <= 1500:
        Cs, hks = cand_from_values(S, t, Y); _, bs = certify(S, Cs, hk=hks, rounds=0)
        wl = bs.get("where") if isinstance(bs.get("where"), list) else []
        if wl: wh.append("출력 자신의 증명서 한도의 몫(정의): " + "; ".join(
            f"t ∈ [{w_['t'][0]:.4g}, {w_['t'][1]:.4g}] {'모든 성분' if w_['component'] == 'all' else '성분 ' + str(w_['component'])} 마디 {w_['node']} {w_['term']} {w_['share']:.2f}" for w_ in wl[:top]))
    if impl_where is not None:
        k_, bn = impl_where
        wh.insert(0, f"구현 에러가 처음 드러난 자리: 걸음 {k_}(t = {t[min(k_, NS)]:.4g}), 관계식 차이가 상한을 넘는 성분·마디 " +
                  ", ".join(f"성분 {j // Nn} 마디 {j % Nn}" for j in bn) + "(반례)")
    rep["어디서"] = dict(status="증명·정의·반례" if impl_where else "증명·정의", text="; ".join(wh) if wh else "오차 없음")
    # ---------------- 왜: pi (length vs angle, enclosed) and i (amplification on the frozen linear problem)
```

```
    why = []; why_status = "증명"
    if np.isfinite(err[kw]) and err[kw] > 0:
        Lp, Ap = _pi_split(sw * Y[kw], sw * refv[kw], beta[kw], err[kw], d)
        if Lp[0] > Ap[1]: kind = "길이(크기의 감쇠·증폭)가 각도보다 크다(증명)"
        elif Ap[0] > Lp[1]: kind = "각도(방향·위상의 어긋남)가 길이보다 크다(증명)"
        else:
            wdt = max(Lp[1], Ap[1]) - min(Lp[0], Ap[0])
            kind = (f"길이와 각도는 판정 해상도 안에서 같은 크기이다: 두 구간이 모두 [{min(Lp[0], Ap[0])
:.2e}, {max(Lp[1], Ap[1]):.2e}] 안에 있어 "
                    f"|길이² − 각도²| ≤ {wdt:.2e}이다(증명; 기준해를 최대한 세분한 뒤의 해상도)")
        why.append(f"π: ‖e‖² = 길이² + 각도², 길이² ∈ [{Lp[0]:.2e}, {Lp[1]:.2e}], 각도² ∈ [{Ap[0]:.2e},
{Ap[1]:.2e}] — {kind}")
    method, ords = None, None
    if claim is not None and not refuted_claim and claim[0] in ("rk", "lmm", "vbdf"):
        method = claim if claim[0] != "lmm" or claim[3] != "tau" else claim[:3] + (tau,)
        if claim[0] == "vbdf": ords = check_vbdf(S, t, Y, claim[1], tau if claim[2] == "tau" else float(
claim[2]))[1]
    elif use_library:
        for nm, cl in library(extended_library).items():
            if any(c_.startswith(nm) for c_ in consistent):
                method = cl
                if cl[0] == "vbdf": ords = check_vbdf(S, t, Y, cl[1], tau)[1]
                break
    k_i = int(bad[0]) if len(bad) else kw
    if step is not None:      # same step size as the original run
        def bb_solver(F_, y0_, Te_, _h=T / N):
            n_ = max(1, int(round(Te_ / _h))); Yb = [y0_.copy()]
            for _ in range(n_): Yb.append(np.asarray(step(F_, Yb[-1], _h), float))
            return np.arange(n_ + 1) * _h, np.array(Yb)
    else:
        bb_solver = solver
    if refuted_claim:
        amp_st, amp = "증명", "i: 구현 에러가 반례로 확정되었으므로 증폭은 원인으로 따지지 않는다"
    elif t_bad is None:
        amp_st, amp = "증명", "i: 정확도 에러가 없으므로 증폭을 원인으로 따질 필요가 없다"
    elif method is not None:
        amp_st, amp = _amplification(S, t, Y, k_i, method, ords, horizon=max(kw, k_i) + 1)
    else:
        amp_st, amp = _amplification_blackbox(S, bb_solver, t, Y, k_i, float(t[max(kw, k_i)]), sw * (Y[k
w] - refv[kw]))
    why.append(amp)
    if amp_st == "반례": why_status = "증명·반례"
    if bs.get("dominant") or (isinstance(bs.get("where"), list) and bs["where"]):
        why.append(f"적분 부등식: 출력 자신의 증명서에서 지배 항은 {bs.get('dominant') or bs['where'][0]
['term']}(정의)")
    rep["왜"] = dict(status=why_status, text="; ".join(why) if why else "오차 없음")
    # ---------------- 어떻게: prescriptions, re-verified when a factory is given
    how = []
    if refuted_claim:
        k_, bn = impl_where
        how.append(f"걸음 {k_}의 상태에서 주장한 공식을 한 걸음 계산해 대상의 출력과 성분 {', '.join(f'{
j // Nn}/마디 {j % Nn}' for j in bn)}를 대조하면 어긋남이 재현된다(반례의 재현); 그 걸음의 단계 계산을
공식과 맞춘 뒤 다시 판정한다")
    if t_bad is not None and not refuted_claim and factory is not None and step is not None:
        n_try = NS
        for _ in range(6):
            n_try *= 2; s2 = factory(n_try); t2, Y2 = s2(F, S.u0.copy(), T)
            ok2 = _proved_within(S, t2, np.asarray(Y2), Cr, br, nref, hr, eps)
            if ok2: how.append(f"걸음을 {n_try}개로 늘려 다시 판정하면 [0, T]에서 오차 ≤ {eps:g}가 증명
된다(처방 확인)"); break
        else:
            how.append(f"걸음을 {n_try}개까지 늘려도 증명이 닫히지 않는다(반례: 이 처방으로는 고쳐지지
않는다)")
    if t_bad is not None and not refuted_claim and factory is not None and step is None:
        for j in range(1, 7):
            s2 = factory(10.0 ** -j); t2, Y2 = s2(F, S.u0.copy(), T)
            if _proved_within(S, np.asarray(t2, float), np.asarray(Y2, float), Cr, br, nref, hr, eps):
                how.append(f"걸음 크기·허용치 매개변수를 10^-{j}배로 줄여 다시 판정하면 [0, T]에서 오차
≤ {eps:g}가 증명된다(처방 확인, {len(t2) - 1}걸음)"); break
        else:
            how.append("걸음 크기·허용치 매개변수를 10^-6배까지 줄여도 증명이 닫히지 않는다(반례: 이 처
방으로는 고쳐지지 않는다)")
    how.append(f"e: 테일러 S_{n_ref}을 {nref}걸음으로 쓰면 [0, T]에서 인증된다(적분 부등식, 증명)")
    rep["어떻게"] = dict(status="증명", text="; ".join(how))
    if refuted_claim: vt, vs = "구현 에러: 대상은 주장한 방법이 아니다(반례)", "반례"
    elif t_bad is not None: vt, vs = f"정확도 에러: 오차가 t = {t_bad:.4g}부터 {eps:g}를 넘는다(반례)",
"반례"
    elif k_ok == NS + 1: vt, vs = f"에러 없음: [0, T]에서 오차 ≤ {eps:g}(증명)", "증명"
    else:
        kk = int(np.argmax((lo <= eps) & (hi > eps)))
```

```
        vt, vs = (f"허용치 경계: 기준해를 최대한 세분해도 t = {t[kk]:.4g}의 오차가 [{lo[kk]:.3e}, {hi[kk]:.3e}]이고 이 구간이 ε = {eps:g}를 품는다. "
                  f"곧 오차는 판정 해상도 {hi[kk] - lo[kk]:.1e} 안에서 ε와 같다(증명). 해상도보다 작은 차이는 이 검증기의 판정 대상이 아니다(정의, 5.3절)"), "증명"
    rep["판정"] = dict(status=vs, text=vt)
    return rep


def _pi_split(y_, r_, b_, e_, d):
    """Prop 1.4 enclosures: length^2 and angle^2 of e = y - u, with ||u - r|| <= b_, e_ = fl(||y - r||)"""
    gq = gam(d + 6)
    nyv = float(np.linalg.norm(y_)); ny = (nyv * (1 - gq), nyv * (1 + gq)); nr = float(np.linalg.norm(r_))
    nu = (max(nr * (1 - gq) - b_, 0), nr * (1 + gq) + b_)
    ne = (max(e_ * (1 - gq) - b_, 0), e_ * (1 + gq) + b_)
    Lp = (max(0.0, ny[0] - nu[1], nu[0] - ny[1]) ** 2, max(ny[1] - nu[0], nu[1] - ny[0]) ** 2)
    Ap = (max(ne[0] ** 2 - Lp[1], 0.0), max(ne[1] ** 2 - Lp[0], 0.0))
    return Lp, Ap


def _expected_checks(claim, NS):
    """number of steps at which the claimed relation applies (one-step: every step; k-step: every window)"""
    if claim[0] == "lmm": return NS + 1 - (len(claim[1]) - 1)
    return NS


def _orders_txt(orders):
    from collections import Counter
    c = Counter(orders); return ", ".join(f"{q}차 {n}걸음" for q, n in sorted(c.items()))


def _proved_within(S, t, Y, Cr, br, nref, hr, eps):
    from six_w import _plain_level, _horner_at
    sw = S.sw; d = len(sw)
    for tt, y in zip(t, Y):
        k = min(int(tt / hr), nref - 1); rv = _horner_at(Cr[k], tt - k * hr)
        if not np.all(np.isfinite(y)): return False
        hround = float(np.linalg.norm(sw * gam(2 * len(Cr[k]) + 4) * sum(np.abs(Cr[k][j]) * abs(tt - k * hr) ** j for j in range(len(Cr[k])))))
        if float(np.linalg.norm(sw * (y - rv))) * (1 + gam(d + 6)) + (_plain_level(br, tt) + hround) * PADB > eps: return False
    return True


def _amplification(S, t, Y, k_state, method, orders=None, horizon=None):
    """i: does the identified method itself amplify where the exact flow cannot?  Frozen linear problem in weighted
    coordinates x' = Jw x, Jw = W J(y) W^-1 the STORED matrix (an exactly defined object), mu a rigorous upper bound of
    its log-norm, so ||e^{tJw} x0|| <= e^{t mu} ||x0||.  The method (Runge-Kutta tableau, fixed-step multistep or
    variable-coefficient BDF), with the scheme's ACTUAL steps (and orders), is applied in exact arithmetic to the exact
    starting values; floating-point iterates are carried with rigorous error bounds:
        RK      e_{n+1} <= Rabs(h ||Jw||) e_n + ||B_n||                    (B_n from rk_step_bound)
        explicit multistep   e_{n+k} <= (sum|alpha_i| + h ||Jw|| sum|beta_i|) / |alpha_k| max e + rho_n
        implicit (BDF, VBDF) ||M^{-1}|| <= 1 / (a - c mu) for M = a I - c Jw, a > c mu
    The exact iterate w_n then satisfies ||w_n|| >= ||x_n|| (1 - gam) - e_n.  If this exceeds e^{t_n mu} ||x0|| the
    method amplifies beyond the exact flow (counterexample, 반례); if the computed run stays below the bound minus the
    error, nothing is claimed beyond that (증명 of the observed inequality)."""
    from rig import expm_rig, nb
    m, Nn = S.m, S.N; d = m * Nn; W = S.sw
    ys = Y[max(k_state - 1, 0)] if np.all(np.isfinite(Y[max(k_state - 1, 0)])) else S.u0
    J = S.Jnum(ys.reshape(m, Nn)); Jw = (W[:, None] * J) / W[None, :]
    Sym = 0.5 * (Jw + Jw.T); mu = lmax_sym_rig(Sym) + gam(4) * nb(Sym)
    nJ = nb(Jw); absJ = np.abs(Jw); g = lambda v: gam(d + 4) * (absJ @ np.abs(v))       # rounding of Jw @ v
    rng = np.random.default_rng(1); x0 = rng.standard_normal(d); n0 = float(np.linalg.norm(x0)) * (1 + gam(d + 4))
    nrm = lambda v: float(np.linalg.norm(v))
    lin = _LinSys(Jw, np.ones(d)); Fl = lambda v: Jw @ v; kind = method[0]; hs = np.diff(t)
    flow = lambda tt: math.exp(min(mu * tt, 700)) * n0 * (1 + EPS_AMP) * (1 + 4 * U)
    def verdict(tn, lo_w, xn, en):
        if lo_w > flow(tn):
            return ("반례", f"i: 얼린 선형 문제(로그노름 μ = {mu:.3g}, 정확한 흐름 ‖x(t)‖ ≤ e^(tμ)‖x0‖, 증명)에서 확인된 방법을 실제 걸음으로 정확한 산술로 적용하면 "
```

```
                    f"t = {tn:.4g}에서 ‖w‖ ≥ {lo_w:.3g} > {flow(tn):.3g}이다(반례: 방법이 정확한 흐름에 없는 증폭을 만든다, 불안정)")
        return None
    e = 0.0; x = x0.copy(); horizon = len(hs) if horizon is None else max(1, min(horizon, len(hs)))
    I_ = np.eye(d); fro = lambda A_: float(np.linalg.norm(A_))
    def s_up(A_):          # rigorous upper bound of ||A_||_2 for the stored matrix
        G_ = A_.T @ A_; l2 = lmax_sym_rig(0.5 * (G_ + G_.T)) + gam(d + 4) * nb(A_) ** 2 + gam(4) * nb(G_)
        return min(nb(A_), math.sqrt(max(l2, 0.0)) * (1 + gam(4)))
    def s_lo(A_):          # rigorous lower bound of ||A_||_2: ||A_ v|| / ||v|| for the top singular vector v
        v = np.linalg.svd(A_)[2][0]; Av = A_ @ v
        return max(0.0, (nrm(Av) - gam(d + 2) * nrm(np.abs(A_) @ np.abs(v))) * (1 - gam(d + 4)) / (nrm(v) * (1 + gam(d + 4))))
    def inv_up(M_):        # rigorous upper bound of ||M_^{-1}||_2:  M^{-1} = Z (M Z)^{-1},  ||(MZ)^{-1}|| <= 1/(1 - ||I - MZ||)
        Z = np.linalg.inv(M_); Rr = I_ - M_ @ Z
        r = s_up(Rr) + fro(gam(d + 4) * (np.abs(M_) @ np.abs(Z) + 1.0 * I_))
        return s_up(Z) / (1 - r) if r < 1 else math.inf
    flow1 = lambda tt: math.exp(min(mu * tt, 700)) * (1 + EPS_AMP) * (1 + 4 * U)
    state = {"all_in": True, "worst": (0.0, 0.0, 0.0, 0.0)}   # (t, lo, up, flow) at the largest up/flow
    def judge(tn, Mh, en, cap=math.inf):
        lo_, up_ = s_lo(Mh) - en, min(s_up(Mh) + en, cap)   # cap: product of step-map norm bounds (submultiplicativity)
        if lo_ > flow1(tn):
            return ("반례", f"i: 얼린 선형 문제(로그노름 μ = {mu:.3g}, 정확한 흐름 ‖e^(tJ)‖ ≤ e^(tμ), 증명)에서 확인된 방법을 실제 걸음으로 정확한 산술로 적용한 "
                    f"사상의 노름이 t = {tn:.4g}에서 ‖M‖ ≥ {lo_:.3g} > {flow1(tn):.3g}이다(반례: 방법이 정확한 흐름에 없는 증폭을 만든다, 불안정)")
        if up_ > flow1(tn): state["all_in"] = False
        if up_ / flow1(tn) > state["worst"][2] / max(state["worst"][3], 1e-300): state["worst"] = (tn, lo_, up_, flow1(tn))
        return None
    def closing(tend):
        if state["all_in"]:
            return ("증명", f"i: 확인된 방법을 실제 걸음으로 얼린 선형 문제에 적용한 사상의 노름이 t = {tend:.4g}까지 모든 시작값에 대해 정확한 흐름의 한계 e^(tμ)(1 + 10⁻¹⁰) 안이다(μ = {mu:.3g}, 증명): 이 구간의 에러는 증폭 때문이 아니다")
        tw, lw, uw, fw = state["worst"]
        return ("증명", f"i: 사상의 노름이 t = {tw:.4g}에서 [{lw:.6g}, {uw:.6g}]이고 정확한 흐름의 한계는 {fw:.6g}이다: 증폭은 많아야 {max(uw / fw - 1, 0):.1e}배이고 "
                f"반례가 될 만큼(정의의 여유 10⁻¹⁰ 초과)은 아니다(증명; 판정 해상도 안에서 흐름과 같은 크기)")
    if kind == "rk":
        # Runge-Kutta: the step map R(h Jw) built on the identity (rk_step_bound gives column error bounds), and the
        # product M_n = R_{n-1} ... R_0 carried as a matrix with a rigorous Frobenius error bound -> holds for every x0
        A, b = np.asarray(method[1], float), np.asarray(method[2], float); cache = {}
        def Rmat(h):
            if h not in cache:
                cols, errs = [], []
                for j in range(d):
                    ej = np.zeros(d); ej[j] = 1.0; yh, B = rk_step_bound(lin, A, b, ej, h, Fl); cols.append(yh); errs.append(B)
                Rh = np.array(cols).T; eR = float(np.linalg.norm(np.array(errs))) * (1 + gam(d * d + 4))
                cache[h] = (Rh, eR, s_up(Rh) + eR)
            return cache[h]
        Mh = I_.copy(); en = 0.0; n_end = min(len(hs), horizon); prod = 1.0
        for n in range(n_end):
            Rh, eR, rn = Rmat(hs[n])
            Mn = Rh @ Mh
            en = rn * en + eR * fro(Mh) + fro(gam(d + 2) * (np.abs(Rh) @ np.abs(Mh)))
            Mh = Mn; prod *= rn
            if not np.all(np.isfinite(Mh)) or not math.isfinite(en): return closing(t[n])
            v = judge(t[n + 1], Mh, en, prod)
            if v: return v
        return closing(t[n_end])
    # multistep methods: carried as OPERATORS (the identity as starting data), so the result holds for every x0.
    # Starting window: exact flow samples e^{t_i Jw} (expm_rig).  Errors in the Frobenius norm (>= the 2-norm).
    if kind in ("lmm", "vbdf"):
        if kind == "lmm":
            al, be = np.asarray(method[1], float), np.asarray(method[2], float); k = len(al) - 1
            if np.max(np.abs(hs - hs[0])) > 8 * U * t[-1]: return ("정의", "i: 걸음이 고르지 않아 고정 걸음 관계식이 적용되지 않는다(이 방법은 이 실행의 방법이 아니다)")
            h = hs[0]; X, eX = expm_rig(h * Jw); nX = nb(X) + eX
            win = [I_.copy()]; ew = [0.0]
            for i in range(1, k):
```

```
                Wi = X @ win[-1]; ew.append(nX * ew[-1] + eX * fro(win[-1]) + fro(gam(d + 4) * (np.abs(X
) @ np.abs(win[-1])))); win.append(Wi)
            implicit = be[-1] != 0
            if implicit:
                a_, c_ = al[-1], h * be[-1]
                M = a_ * I_ - c_ * Jw
                Minv = min(1.0 / (a_ - c_ * mu) if a_ - c_ * mu > 0 else math.inf, inv_up(M))
                if not math.isfinite(Minv): return closing(t[0])
            amp_hist = (np.sum(np.abs(al[:-1])) + h * nJ * np.sum(np.abs(be[:-1])))
            for n in range(min(len(t) - k, horizon)):
                rhs = -sum(al[i] * win[n + i] - h * be[i] * (Jw @ win[n + i]) for i in range(k))
                rr = gam(2 * k + d + 8) * sum(abs(al[i]) * np.abs(win[n + i]) + h * abs(be[i]) * (absJ @
 np.abs(win[n + i])) for i in range(k))
                ein = amp_hist * max(ew[n:n + k])
                if implicit:
                    Xn = np.linalg.solve(M, rhs); res = M @ Xn - rhs
                    en = Minv * (ein + fro(rr) + fro(res) + fro(gam(d + 4) * (np.abs(M) @ np.abs(Xn) + n
p.abs(rhs))))
                else:
                    Xn = rhs / al[-1]; en = (ein + fro(rr) + gam(2) * fro(Xn)) / abs(al[-1])
                win.append(Xn); ew.append(en)
                if not np.all(np.isfinite(Xn)) or not math.isfinite(en): return closing(t[n + k - 1])
                v = judge(t[n + k], Xn, en)
                if v: return v
            tend = t[min(len(t) - 1, horizon + k - 1)]
        else:
            if not orders: orders = check_vbdf(S, t, Y, method[1], method[2] if not isinstance(method[2]
, str) else 0.0)[1]
            win = [I_.copy()]; ew = [0.0]; n_end = min(len(t), len(orders) + 1, horizon + 1)
            for n1 in range(1, n_end):
                q = orders[n1 - 1]; wts = _dweights(t[n1 - q:n1 + 1]); a_ = wts[-1]
                rhs = -sum(wts[i] * win[n1 - q + i] for i in range(q)); M = a_ * I_ - Jw
                Minv = min(1.0 / (a_ - mu) if a_ - mu > 0 else math.inf, inv_up(M))
                if not math.isfinite(Minv): return closing(t[n1 - 1])
                Xn = np.linalg.solve(M, rhs); res = M @ Xn - rhs
                dw = gam(4 * q + 10) * np.abs(wts)
                pert = sum(dw[i] * fro(win[n1 - q + i]) for i in range(q)) + dw[-1] * fro(Xn)
                rres = fro(res) + fro(gam(d + 4) * (np.abs(M) @ np.abs(Xn) + np.abs(rhs))) + fro(gam(q +
 2) * sum(np.abs(wts[i]) * np.abs(win[n1 - q + i]) for i in range(q)))
                en = (sum(abs(wts[i]) * ew[n1 - q + i] for i in range(q)) + pert + rres) * Minv
                win.append(Xn); ew.append(en)
                if not np.all(np.isfinite(Xn)) or not math.isfinite(en): return closing(t[n1 - 1])
                v = judge(t[n1], Xn, en)
                if v: return v
            tend = t[n_end - 1]
        return closing(tend)
    return ("정의", "i: 방법이 확인되지 않음")


def _amplification_blackbox(S, solver, t, Y, k_state, t_end, e_dir):
    """i for a scheme outside the library: the TARGET ITSELF is run on the frozen linear problem y' = J
y (J frozen at
    the state where the error is first refuted).  Its output is exact data; only the norms are rounded.
 Starting values:
    the error direction at the worst time, the top right singular vector of e^{t_end Jw} (computed), and
 a fixed random
    vector.  If some output exceeds the exact-flow bound e^{t mu} ||x0|| (1 + EPS_AMP) -> 반례 (the impl
ementation
    amplifies).  Otherwise the largest measured ratio is a proved lower bound of its amplification on th
ose starting
    values; for a scheme outside the library no claim is made about every starting value (definition of
the scope)."""
    from rig import nb
    m, Nn = S.m, S.N; d = m * Nn; W = S.sw
    ys = Y[max(k_state - 1, 0)] if np.all(np.isfinite(Y[max(k_state - 1, 0)])) else S.u0
    J = S.Jnum(ys.reshape(m, Nn)); Jw = (W[:, None] * J) / W[None, :]
    Sym = 0.5 * (Jw + Jw.T); mu = lmax_sym_rig(Sym) + gam(4) * nb(Sym); g = gam(d + 4)
    from scipy.linalg import expm
    starts = [np.asarray(e_dir, float), np.linalg.svd(expm(t_end * Jw))[2][0], np.random.default_rng(1).
standard_normal(d)]
    worst = (0.0, None)
    for xw in starts:
        if not np.all(np.isfinite(xw)) or np.linalg.norm(xw) == 0: continue
        x0 = xw / W
        with np.errstate(all="ignore"):
            tl, Yl = solver(lambda y: J @ y, x0.copy(), t_end)
        tl = np.asarray(tl, float); Yl = np.asarray(Yl, float); n0 = float(np.linalg.norm(W * x0)) * (1
+ g)
        for tt, yy in zip(tl, Yl):
            if not np.all(np.isfinite(yy)):
```